System Hacking - Study Guide

Concepts, Techniques, and Tools Across the Attack Lifecycle

Ethical Hacking / Network Security Study Notes
November 2026

1. System Hacking Concepts
2. Cracking Passwords
3. Escalating Privileges
4. Executing Applications
5. Hiding Files
6. Covering Tracks
Quick Revision Summary

> ***Scope & Ethics Notice*** *This guide is for educational / certification-study purposes (e.g., CEH-style exam prep). It describes concepts, attacker goals, and categories of tools at a conceptual level so you can recognize, defend against, and discuss these techniques. It is not a step-by-step exploitation manual. Using these techniques against any system without explicit, written authorization is illegal in most jurisdictions.*

1. System Hacking Concepts

1.1 What is System Hacking?

System hacking refers to the stage of an attack where the attacker, having already identified a target (through reconnaissance and scanning), attempts to gain, maintain, and expand **unauthorized access** to a specific system, and then conceals the intrusion. It is the "hands-on-keyboard" phase that follows information gathering and vulnerability identification.

1.2 The CEH System Hacking Goals (Attack Lifecycle)

Classic system-hacking methodology is taught as a five-stage goal sequence, which this guide follows section by section:

Stage	Attacker's Goal
1. **Gaining Access**	Obtain valid credentials or exploit a vulnerability to get onto the system (Section 2: Cracking Passwords)
2. **Escalating Privileges**	Move from a limited account to administrator/root/SYSTEM level (Section 3)
3. **Maintaining Access**	Ensure continued access even after the initial foothold might be lost (Section 4: Executing Applications / planting backdoors)
4. **Hiding Files**	Conceal malicious tools and data from the user and security software (Section 5)
5. **Covering Tracks**	Erase or alter evidence of the intrusion (Section 6)

1.3 Authentication and Why It's the First Target

Most systems gate access behind **authentication** — something you know (password), something you have (token/smartcard), or something you are (biometrics). Because password-based authentication is still the most common control, and because humans choose weak, reused, or guessable passwords, credential attacks remain the most common entry point into the "gaining access" stage.

1.4 Common Authentication Weaknesses Attackers Target

- Weak, default, or reused passwords
- Password hashes stored or transmitted without adequate protection (e.g., unsalted hashes, legacy protocols like LM/NTLMv1)
- No account lockout or rate limiting on login attempts
- No multi-factor authentication (MFA)
- Credentials cached in memory, configuration files, or scripts
- Social-engineered disclosure of credentials

1.5 Types of Password Attacks (Overview)

Category	Description
Non-electronic	Shoulder surfing, social engineering, dumpster diving — no technical tool involved

Passive online	Sniffing credentials in transit, replay attacks, man-in-the-middle
Active online	Dictionary, brute-force, hash injection, and similar attacks directly against a live authentication service
Offline	Attacking stolen password hashes without touching the live system (dictionary, brute-force, rainbow table)
Default password	Trying vendor/factory-default credentials that were never changed

2. Cracking Passwords

2.1 What is Password Cracking?

The process of recovering a password (or an equivalent credential such as a password hash) through methods ranging from guessing to systematic computation, in order to gain unauthorized access to an account or system.

2.2 Password Cracking Techniques

Dictionary Attack

Tries every entry in a precompiled word list (common passwords, leaked-password dumps, or domain-specific terms) against the target. Fast and effective against weak/common passwords but fails against strong, random ones.

Brute-Force Attack

Systematically tries every possible character combination up to a given length. Guaranteed to eventually succeed but can be computationally infeasible against long, complex passwords — time grows exponentially with length and character-set size.

Hybrid Attack

Combines a dictionary with brute-force logic, e.g., appending numbers/symbols to dictionary words (`password123!`) to catch common human password-modification habits.

Rule-Based Attack

Applies transformation rules to a base word list (leetspeak substitutions, capitalization patterns, common suffixes) to generate more realistic guesses than raw brute-force, used heavily by tools like Hashcat.

Rainbow Table Attack

Uses a precomputed table of hash-to-plaintext mappings to reverse a captured hash almost instantly, trading disk space for compute time. Defeated by **salting** (adding a unique random value to each password before hashing).

Password Guessing

Manually or automatically trying likely passwords based on knowledge of the target (birthdays, pet names, default vendor passwords) — effective because people frequently choose personally meaningful passwords.

Phishing / Social Engineering

Tricking the user into directly revealing their password (fake login pages, pretexting phone calls) — often the path of least resistance compared to any computational attack.

Spraying Attack

Instead of trying many passwords against one account (which triggers lockouts), the attacker tries a small number of common passwords against **many** accounts, staying under per-account lockout thresholds.

2.3 Passive Online Attacks

- **Wire sniffing**: capturing authentication traffic on the network (effective against legacy, unencrypted protocols; see the companion Sniffing study guide).
- **Man-in-the-middle / replay attacks**: intercepting and relaying or reusing captured authentication exchanges.

2.4 Active Online Attacks

- **Password guessing against a live service** (login forms, RDP, SSH) — directly tries credentials and is noisy/detectable via failed-login logs.
- **Hash injection / Pass-the-Hash**: using a captured password *hash* directly to authenticate to NTLM-based services without ever knowing the plaintext password.
- **LLMNR/NBT-NS poisoning**: responding to broadcast name-resolution requests to trick a victim machine into sending its hashed credentials to the attacker.

2.5 Offline Attacks

Performed against stolen hash files (e.g., exfiltrated from a compromised database or extracted from a system's SAM/shadow file) with no interaction with the live target, so there's no lockout risk and attacks can run for as long as the attacker has compute time.

2.6 Password Hashing Concepts

- **Hashing** converts a password into a fixed-length value that (ideally) cannot be reversed, so systems store the hash rather than the plaintext.
- **Salting** adds a unique random value per password before hashing, defeating precomputed rainbow tables and ensuring identical passwords produce different stored hashes.
- Legacy hash types (e.g., **LM hash**) are weak (case-insensitive, split into small chunks) and crack far faster than modern ones (e.g., **bcrypt**, **Argon2**, salted SHA-256/512) which are intentionally slow/memory-hard to resist brute-forcing.

2.7 Password-Cracking Tools (Overview)

Tool	Brief Description
John the Ripper	Open-source, highly configurable password cracker supporting dictionary, brute-force, and rule-based attacks across many hash formats; widely used CLI standard.
Hashcat	GPU-accelerated password recovery tool; extremely fast against most hash types via dictionary, brute-force, rule-based, and hybrid attacks.
Cain & Abel	Windows GUI suite combining packet capture, hash extraction, and cracking (dictionary/brute-force/rainbow table) with its own sniffer (also used for ARP poisoning).
Ophcrack	Rainbow-table-based cracker specifically for Windows LM/NTLM hashes; often run from bootable media.
L0phtCrack	Commercial Windows password auditing tool combining multiple cracking methods, historically used for AD password-policy audits.
Hydra (THC-Hydra)	Fast network login-cracker supporting brute-force/dictionary attacks against many live services (SSH, FTP, RDP, HTTP forms, etc.).
Medusa	Parallel, modular network login-cracker similar in purpose to Hydra.
RainbowCrack	Generates and uses rainbow tables for fast offline hash reversal.

2.8 Defenses Against Password Cracking

- Enforce long, complex, unique passwords; prefer passphrases over short complex strings.
- Use strong, salted, slow hashing algorithms (bcrypt, scrypt, Argon2, PBKDF2).
- Enforce account lockout / progressive delay after failed attempts, with spray-attack-aware thresholds.
- Require multi-factor authentication wherever possible.
- Disable legacy protocols (LM hashing, NTLMv1, SMBv1) that are weak by design.

- Monitor authentication logs for brute-force/spray patterns (many failures across many accounts, or from one source).
- Educate users on phishing recognition and password hygiene (password managers, no reuse).

3. Escalating Privileges

3.1 What is Privilege Escalation?

The process of exploiting a bug, design flaw, misconfiguration, or weak permission setting to gain elevated access to resources that are normally protected from a given user/account — moving from a low-privileged foothold to administrator, root, or SYSTEM-level control.

3.2 Types of Privilege Escalation

Vertical Privilege Escalation

A lower-privileged user gains higher privileges (e.g., a standard user obtains Administrator/root/SYSTEM rights). This is the classic, most impactful form, as it typically grants full control of the host.

Horizontal Privilege Escalation

An attacker with one user's level of access gains access to the resources/data of a **different** user at the same privilege level (e.g., accessing another customer's account in a web application due to an authorization flaw), without necessarily gaining admin rights.

3.3 Common Privilege Escalation Techniques (Conceptual Categories)

Technique Category	Description
Exploiting unpatched vulnerabilities	Using a known kernel or OS-level vulnerability (CVE) for which a patch exists but hasn't been applied.
Misconfigured file/service permissions	A service running as SYSTEM/root that is writable by a low-privileged user, allowing the attacker to replace its binary or configuration.
Insecure service paths	Unquoted or writable service executable paths that let an attacker plant a malicious binary executed with the service's elevated privileges.
DLL hijacking	Placing a malicious DLL where a privileged application will load it instead of the legitimate library, due to insecure search-path behavior.
Access token manipulation	Abusing Windows access tokens (e.g., impersonation privileges) to act with another, more privileged security context.
Credential harvesting on the host	Extracting cached credentials, stored passwords, or tokens left in memory, files, or the registry of the compromised machine to reuse elsewhere.
Sudo/SUID misconfiguration (Linux)	Overly permissive `sudoers` rules, or SUID-root binaries that can be abused to execute arbitrary commands as root.
Kernel exploits	Exploiting a bug in OS kernel code itself to execute attacker code at the highest privilege level.
Bypassing User Account Control (UAC)	Techniques that trick Windows into auto-elevating a process without the expected user consent prompt.

3.4 Why Privilege Escalation Matters

Many initial access vectors (phishing, exploiting a web app) only grant a limited foothold. Escalation is often the step that converts a minor compromise into full system or domain control, enabling data theft, further lateral movement, and persistent control.

3.5 Privilege Escalation Tools (Overview)

Tool	Brief Description
PowerUp / PowerSploit	PowerShell modules that automatically enumerate common Windows privilege-escalation misconfigurations (weak service permissions, unquoted paths, etc.).
WinPEAS	Automated Windows enumeration script that highlights likely privilege-escalation vectors on a compromised host.
LinPEAS	The Linux counterpart to WinPEAS — enumerates SUID binaries, sudo misconfigurations, cron jobs, and more.
Metasploit (local exploit/post modules)	Framework with built-in modules for known local privilege-escalation exploits and post-exploitation enumeration.
Mimikatz	Extracts credentials, hashes, and Kerberos tickets from Windows memory, often used after an initial foothold to pivot to higher-privileged accounts.
BloodHound	Maps Active Directory relationships/permissions to reveal non-obvious privilege-escalation paths within a domain.

3.6 Defenses Against Privilege Escalation

- Apply security patches promptly, especially for kernel and OS-level vulnerabilities.
- Follow least-privilege principles for services and scheduled tasks (run as a dedicated low-privilege account, never SYSTEM/root unless required).
- Audit file/folder and service permissions regularly; fix writable binaries/paths owned by privileged services.
- Keep UAC enabled at a strict setting; avoid auto-elevation of untrusted software.
- Harden `sudoers` configuration and audit SUID binaries on Linux systems.
- Use endpoint detection (EDR) to flag known escalation tool behavior (e.g., Mimikatz-style credential dumping).
- Apply Credential Guard / LSA protection on Windows to hinder in-memory credential extraction.

4. Executing Applications

4.1 What is “Executing Applications” in This Phase?

Once an attacker has access (and ideally elevated privileges), the next goal is typically to **run their own tools** on the compromised system — to maintain access, pivot further into the network, exfiltrate data, or achieve their ultimate objective. This stage covers remote command/code execution and the planting of persistence mechanisms.

4.2 Typical Objectives at This Stage

- **Remote Command/Code Execution (RCE)**: running attacker-chosen commands or code on the target.
- **Deploying backdoors / RATs**: installing tools for ongoing remote access (see the companion Malware study guide, Section 2).
- **Keylogging**: capturing keystrokes to harvest further credentials or sensitive data over time.
- **Spyware deployment**: covertly monitoring user activity, screen, clipboard, or files.
- **Lateral movement**: using the current foothold to reach other systems on the network.
- **Living off the land**: using legitimate, already-installed administrative tools (PowerShell, WMI, PsExec, scheduled tasks) to execute code, which blends in with normal administrative activity and is harder to flag than a dropped new binary.

4.3 Common Execution Techniques (Conceptual)

Technique	Description
Scheduled tasks / cron jobs	Creating a task that runs attacker code on a schedule or at startup.
Remote service creation	Installing a new Windows service (or using `PsExec`-style tools) to execute a command on a remote host with appropriate credentials.
WMI (Windows Management Instrumentation)	A built-in Windows management interface that can be abused to execute code remotely without dropping new files.
Scripting engines	PowerShell, VBScript, or `mshta` / `regsvr32` abused to run attacker code using only trusted, signed OS binaries.
Macro-enabled documents	Office macros that execute code when a user opens a crafted document (often the initial-access step, but relevant here as an execution vector too).

4.4 Keyloggers and Spyware (Overview)

- **Hardware keyloggers**: physical devices placed between a keyboard and a computer, capturing keystrokes independent of the OS.
- **Software keyloggers**: run as a program/driver on the OS, intercepting keyboard input at the kernel or application level.
- **Spyware**: broader category also capturing screenshots, clipboard content, browser history, webcam/microphone feeds, often bundled with other malware.

4.5 Execution / Post-Exploitation Tools (Overview)

Tool	Brief Description
Metasploit Framework	The standard exploitation/post-exploitation framework; includes `meterpreter`, an in-memory agent for command execution, file transfer, and pivoting after initial access.
PsExec (Sysinternals)	Legitimate Microsoft tool for running processes on remote Windows systems; frequently abused by attackers (and red teamers) for lateral movement using valid admin credentials.

PowerShell Empire / Covenant	Post-exploitation C2 frameworks built around PowerShell/.NET “living-off-the-land” execution techniques.
Cobalt Strike	Commercial red-team C2 platform providing a “Beacon” agent for command execution, lateral movement, and persistence (also frequently abused by real attackers).
Mimikatz	(Also listed in Section 3) used here to harvest credentials that enable further remote execution via legitimate admin channels.
Netcat	Lightweight networking utility often used to set up simple remote shells (listener/connector) for command execution.

4.6 Defenses Against Malicious Execution

- Restrict and monitor PowerShell/WMI use via logging (PowerShell Script Block Logging, Constrained Language Mode) and application control policies.
- Use application allow-listing (e.g., Windows Defender Application Control, AppLocker) to block unauthorized executables/scripts.
- Disable or tightly restrict remote-execution tools (PsExec-equivalents, WMI, RDP) to only what’s operationally needed, with MFA.
- Monitor for anomalous parent-child process relationships (e.g., Office apps spawning shells).
- Deploy EDR to detect known C2/post-exploitation frameworks’ behavioral signatures.
- Network segmentation to limit how far lateral movement can reach after an initial compromise.

5. Hiding Files

5.1 Why Attackers Hide Files

Once tools are planted and running, attackers need them to remain undetected by the user and by security software for as long as possible, to preserve access and avoid triggering an incident response.

5.2 Techniques for Hiding Files (Conceptual)

Rootkits

Software designed to hide the existence of certain processes, files, network connections, or registry entries from the operating system and security tools themselves, by intercepting and manipulating system calls or operating at the kernel level. Categories include: - **User-mode rootkits**: operate at the application level, hooking API calls. - **Kernel-mode rootkits**: operate with OS-kernel privileges, far harder to detect, as they can lie to any tool that queries the OS. - **Bootkits**: infect the boot process itself, loading before the OS and its security software. - **Firmware/hypervisor rootkits**: hide below the OS entirely (BIOS/UEFI or a malicious hypervisor), the hardest category to detect from within the running OS.

NTFS Alternate Data Streams (ADS)

A feature of the NTFS file system allowing a file to have more than one data stream. Data hidden in an ADS (`filename.txt:hidden.exe`) doesn't appear in normal file size/listing in Explorer, making it a classic Windows file-hiding technique.

Steganography

Concealing data (including executables or scripts) inside the data of another, innocuous-looking file — commonly an image, audio, or video file — so that the carrier file appears completely normal under casual inspection.

Simple OS-Level Hiding

- Setting the **Hidden** and **System** file attributes (`attrib +h +s`) to keep files out of default Explorer views.
- Naming files to closely resemble legitimate system files, or placing them in deep/unusual but plausible-looking directories.
- Timestomping: altering a file's creation/modification timestamps to blend in with legitimate system files of a similar age.

5.3 File/Data-Hiding Tools (Overview)

Tool	Brief Description
Rootkit frameworks (e.g., historical examples like TDSS, ZeroAccess)	Studied conceptually as examples of kernel-mode concealment techniques; not legitimate utilities.
OpenStego / Steghide	Tools for embedding and extracting hidden data within image/audio carrier files using steganographic techniques.
NTFS streams utilities	Built-in `dir /r` (lists ADS) and tools like Sysinternals' **Streams.exe** to detect/enumerate alternate data streams.
GMER / TDSSKiller / RootkitRevealer	Anti-rootkit scanners that compare high-level and low-level system views to reveal hidden kernel-mode artifacts.

5.4 Defenses Against File-Hiding Techniques

- Use file-integrity monitoring (FIM) to detect unexpected new/modified files, including in alternate data streams.
- Regularly scan for alternate data streams on sensitive systems (`dir /r`, Streams.exe, or EDR products that

inspect ADS by default).

- Use anti-rootkit tools and offline/bootable scanning (scanning a disk from outside the potentially-compromised running OS) to bypass a kernel-mode rootkit’s deception.
- Maintain Secure Boot and verify firmware/boot-chain integrity to resist bootkits.
- Baseline and monitor cryptographic hashes of critical system files to catch unauthorized replacements.

6. Covering Tracks

6.1 Why Attackers Cover Tracks

The final goal of the classic system-hacking lifecycle: erase or alter evidence of the intrusion so defenders don't realize a compromise occurred, can't determine how it happened, and can't attribute it, extending the attacker's window of access and avoiding consequences.

6.2 Common Track-Covering Techniques (Conceptual)

Technique	Description
Clearing event logs	Deleting or selectively editing Windows Event Logs / Linux `syslog` / `auth.log` entries that record the attacker's logons, commands, or actions.
Disabling auditing/logging	Turning off audit policies or logging services before carrying out further actions, so nothing is recorded in the first place.
Timestomping	Altering file creation/modification/access timestamps so malicious files blend in with legitimate, older system files.
Clearing command history	Removing shell history files (e.g., `.bash_history`) or PowerShell history to hide commands that were run.
Manipulating or deleting specific log entries	More surgical than wholesale log clearing (which is itself a red flag) — removing only the entries that reference the attacker's activity.
Using legitimate/trusted processes	Performing actions through living-off-the-land binaries so that log entries, if they exist, look like ordinary administrative activity.
Disabling or evading security software	Terminating or reconfiguring antivirus/EDR agents so later activity isn't logged, flagged, or blocked, and historical detections are also hidden if possible.
Tunneling/encrypting C2 traffic	Disguising outbound command-and-control traffic (e.g., inside DNS queries or HTTPS) to blend in with normal traffic and evade network monitoring.

6.3 Anti-Forensic Concepts

Anti-forensics refers more broadly to any technique that impedes forensic investigation — not just hiding an intrusion, but also disrupting the investigator's ability to reconstruct events, attribute the attack, or even prove an intrusion took place at all. This includes data wiping (secure deletion that resists file-recovery tools), encryption of attacker tools/data, and exploiting gaps/limitations in forensic tools themselves.

6.4 Track-Covering / Anti-Forensic Tools (Overview)

Tool	Brief Description
CCleaner / BleachBit	Legitimate system-cleanup utilities that can also be abused to wipe browser history, temp files, and some logs.
Event Viewer / wevtutil (built-in)	Native Windows utilities (`wevtutil cl <logname>`) that can clear specific event logs — a capability intended for legitimate administration but commonly abused.
Meterpreter's `clearev` / timestomp modules	Built into common post-exploitation frameworks specifically to clear Windows event logs and alter file timestamps in one step.
Secure-deletion utilities (e.g., `shred`, `sdelete`)	Overwrite file contents before deletion to resist standard file-recovery/undelete tools.

6.5 Defenses Against Track-Covering

- **Forward logs off-host in real time** to a centralized, append-only SIEM/log server so local log deletion doesn’t erase the evidence trail.
- Enable and protect **audit log integrity** (write-once storage, log signing) so tampering is itself detectable.
- Monitor specifically for **log-clearing events** (e.g., Windows Event ID 1102 “audit log cleared”) as a high-confidence indicator of compromise.
- Use EDR/host-based monitoring that captures activity independent of the logs an attacker might clear.
- Maintain offline, immutable backups of logs and critical data for forensic reconstruction after an incident.
- Regularly test incident-response and forensic-readiness procedures (log retention policy, chain-of-custody practices) before they’re needed.

Quick Revision Summary

#	Stage	One-line takeaway	Key Tool Examples
1	System Hacking Concepts	Five-stage lifecycle: access, escalate, maintain, hide, cover tracks.	—
2	Cracking Passwords	Dictionary/brute-force/hybrid/rainbow-table attacks; defend with salted hashing + MFA.	John the Ripper, Hashcat, Hydra
3	Escalating Privileges	Vertical (gain higher rights) vs. horizontal (access peer's data); patch + least privilege.	PowerUp, WinPEAS/LinPEAS, Mimikatz
4	Executing Applications	Attacker runs code/tools on the host, often via living-off-the-land techniques.	Metasploit/Meterpreter, PsExec, Cobalt Strike
5	Hiding Files	Rootkits, ADS, steganography, timestomping conceal attacker artifacts.	Rootkits, Steghide, Streams.exe (detection)
6	Covering Tracks	Clear/alter logs and timestamps to erase evidence; defend with centralized, tamper-evident logging.	wevtutil, Meterpreter clearev, BleachBit